

XPLOR — Canonical Platform Specification (Golden Document)

Status: Authoritative

This document defines the **non-negotiable foundation, capabilities, and evolution path** of XPLOR.

Every feature, subsystem, and future application **must comply with this document**. Any deviation requires explicit architectural approval.

0. Product Identity & Scope

0.1 What XPLOR Is

XPLOR is a **Spatial Intelligence Platform** that transforms 2D spatial inputs into structured 3D environments and immersive VR experiences, while enabling intelligent automation, analysis, and future spatial applications.

XPLOR is **not** merely a 3D editor.

It is a **canonical spatial runtime** upon which editors, AI agents, VR viewers, and future applications operate as clients.

0.2 What XPLOR Is *Not*

- Not a one-click generative interior design tool
 - Not a consumer-only visualization toy
 - Not renderer-dependent
 - Not UI-driven in its core logic
-

1. Core Architectural Principles (Non-Negotiable)

1.1 Single Canonical Spatial Model

- A **single, authoritative spatial data model** represents every scene.
- All edits, AI operations, exports, and renders **must originate from this model**.
- UI state, renderer state, and VR state are **derived views**, never sources of truth.

1.2 Command-Based Mutation

- Every scene change is executed as a **command**.
- Commands are:
 - Deterministic
 - Replayable

- Undoable / Redoable
- Auditable
- No UI component may directly mutate scene state.

1.3 Constraint-First System

- Hard constraints must always be satisfied.
- Soft constraints are optimized but never enforced blindly.
- AI operates **inside** constraints, never around them.

1.4 Renderer Independence

- Renderers consume spatial data but never own it.
- Web, VR, Unreal, AR, and future renderers use adapters over the same model.

1.5 Headless Executability

- The entire platform must function without UI.
- All core logic must be callable via APIs or services.

2. Canonical Spatial Intelligence Core (Foundational Layer)

2.1 Scene Graph & Spatial Schema

The spatial model represents:

- Spaces (rooms, zones)
- Structural elements (walls, doors, windows)
- Objects (furniture, fixtures, assets)
- Relationships (attachments, adjacency, facing)
- Constraints (hard and soft)
- Metadata (units, style, intent)

The schema is: - Versioned - Serializable - Backward-compatible

2.2 Command System

Supported command categories:

- Structural Commands (add wall, modify opening)
- Object Commands (add, move, rotate, scale)
- Material Commands (apply, modify)
- Lighting Commands
- Constraint Evaluation Commands
- AI-Proposed Commands (validated before execution)

2.3 Constraint & Rule Engine

Hard Constraints (Non-violable)

- No geometry overlap

- Minimum walkable clearance
- Door/window accessibility
- Structural alignment
- Human-scale constraints

Soft Constraints (Optimizable)

- Visual balance
- Symmetry
- Focal alignment
- Style coherence
- Layout efficiency

2.4 Evaluation & Scoring Engine

- Scores scenes against defined metrics
 - Enables layout ranking, AI selection, and repair strategies
-

3. Asset Intelligence System

3.1 Asset Definition

Assets are **knowledge objects**, not files.

Each asset includes: - Geometry & LODs - Materials (PBR-compliant) - Physical dimensions - Clearance requirements - Affordances (sit, store, support, etc.) - Anchor rules (wall, floor, free-standing) - Style tags - Room compatibility

3.2 Asset Ingestion

- Native asset library
- User-uploaded assets
- External imports (GLB, FBX)
- Validation against asset schema

3.3 Asset Recommendation Layer

- Context-aware suggestions
 - Room-type and style-based ranking
 - AI-assisted but rule-validated
-

4. Project & Workspace Management

4.1 Authentication & Identity

- User login & signup
- OAuth support
- Workspace-based project isolation

4.2 Project Lifecycle

- Create, open, duplicate, archive, delete projects
 - Versioned project history
 - Snapshot & rollback support
-

5. 2D Spatial Authoring Client

5.1 Core 2D Tools

- Wall creation & editing
- Room segmentation
- Doors and windows
- Grid snapping & scale calibration
- Dimension constraints

5.2 Input-Assisted Authoring

- PDF/image floor plan overlay
- Perspective correction
- Manual correction tools

5.3 2D Persistence

- Save canonical spatial data
 - Versioned spatial files
-

6. 2D → 3D Conversion Pipeline

- Deterministic geometry generation
 - Unit-accurate walls, floors, ceilings
 - Openings preserved
 - Structural intent retained
-

7. 3D Editing Client

7.1 Geometry Editing

- Move, rotate, scale objects
- Structural edits synced with canonical model

7.2 Materials & Appearance

- PBR material system
- Editable colors and textures

7.3 Lighting

- Light creation & editing
- Real-time previews

7.4 Assets & Furniture

- Drag-and-drop placement
 - Snapping & alignment
 - Intelligent suggestions
 - Custom asset import
-

8. Rendering & Visualization Layer

8.1 Web Rendering

- Real-time preview
- Performance-optimized rendering

8.2 VR Walkthrough

- Walkable, scale-accurate environments
 - Teleport and smooth locomotion
 - VR state synced with canonical scene
-

9. AI & Automation Layer

9.1 Intelligent Asset Placement

- Rule-seeded layouts
- AI-ranked placement candidates
- Constraint-validated execution

9.2 Scene Optimization & Repair

- Detect constraint violations
- Propose minimal fixes
- Improve layout quality

9.3 Automatic Asset Generation

- External AI-based asset generation
 - Schema validation before ingestion
-

10. Export & Interoperability

- Export canonical scenes to GLB
- Geometry, materials, lighting preserved

- External pipeline compatibility
-

11. Headless Execution & Future Applications

- API-first core
 - Batch processing
 - Simulation and evaluation
 - Third-party application support
-

12. Governance & Evolution

12.1 Versioning

- Scene schema versioning
- Migration paths guaranteed

12.2 Architectural Discipline

- Core principles override feature velocity
 - No feature may bypass the spatial core
-

13. Usage Constraints, Quotas & Cost Controls (Platform Economics Layer)

This section defines **mandatory platform-level constraints** to ensure predictable infrastructure cost, fair usage, and scalable unit economics.

13.1 Job-Based Compute Limits

Definition

A **job** is any compute-intensive backend operation, including: - 2D → 3D conversion - Regeneration of 3D from modified 2D - 3D → VR build - High-quality lighting recomputation - AI-driven asset or layout regeneration

Tier-Based Allocation

- **Lite:** Fixed, low monthly job quota
- **Standard / Pro:** Higher monthly job quota
- **Enterprise:** Contract-defined job quota

Additional Jobs

- Extra jobs may be purchased beyond monthly limits
 - Per-job pricing must directly offset compute and GPU costs
-

13.2 Custom Asset Upload Limits

Definition

Custom assets include: - User-uploaded furniture or decor models - External GLB / FBX assets

Tier-Based Allowances

- **Lite:** Limited free uploads
- **Standard:** Moderate free uploads
- **Pro:** High free upload allowance
- **Enterprise:** Contract-defined limits

Overages

- Additional uploads are billed per asset
 - Upload triggers validation and preprocessing costs
-

13.3 Admin & Collaboration Limits

Definition

Admins are users with full editing, management, and project control privileges.

Constraints

- Default admin count enforced per tier
 - Maximum admin cap per tier
 - Optional paid admin seat expansion where applicable
-

13.4 Screen & Device Limits

Definition

A **screen** refers to: - Active editor displays - VR headsets - Concurrent visualization endpoints

Constraints

- Tier-based default screen limits
 - Hard maximum screen caps
 - Paid expansions allowed (except Enterprise)
-

13.5 Asset Marketplace & Selling Controls

Definition

Assets sold or distributed via the XPLORE marketplace.

Constraints

- Free asset selling quota per tier
 - Per-asset fee or commission beyond free quota
 - Reduced per-item cost at higher tiers
-

13.6 Cost-Control Design Principles

- All heavy compute actions are quota-bound
 - Storage-heavy actions are capped or metered
 - Real-time VR usage is session-aware
 - Enterprise plans operate via negotiated contracts
-

13.7 Enforcement & User Experience Rules

- Hard stops when limits are reached
 - Real-time in-product usage meters
 - Pre-action warnings before quota exhaustion
 - One-click add-on purchase flows
-

13.8 Enterprise Plan Handling

- All limits configurable
 - Custom pricing models
 - Dedicated or isolated infrastructure options
 - SLA-backed usage guarantees
-

Economic Principle:

XPLORE must remain **cost-predictable by design**, not by user goodwill. All compute-heavy or storage-heavy operations are explicit, metered, and monetizable.